SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
Department of Computer Science and Engineering
CSA1301 – Theory of Computation

COURSE ASSIGNMENT REPORT

Design and Develop of a Non-Deterministic Finite Automaton (NFA) for a Smart Parking Gate System

Submitted By

Name: Manohar C

Register No: 192372252

Department of Computer Science and Engineering

Assignment Information

Department:	Computer Science and Engineering
Programme:	B.Tech
Course Code & Course Name	CS301 – Theory of Computation / Automata Theory
Academic Year / Batch	2026–27, IV Year / VII Semester
Faculty Name	Dr.P. Vijayaragavan
Assignment Title	Design of a Non-Deterministic Finite Automaton (NFA) for a Smart Parking Gate System
Date of Issue	01/09/2026
Date of Submission	04/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO3: Design finite automata (DFA/NFA) to recognize languages/behavior of simple systems
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate (Create at design stage)
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation & Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Automated/IoT-based smart parking systems used in smart cities, malls, airports and campuses

Assignment Problem / Challenge

This assignment presents a realistic embedded/computing system-modelling problem drawn from smart-city infrastructure, related to Course Outcome CO3 (design of finite automata for system behavior). Students are required to apply concepts of automata theory rather than reproduce definitions, analyses the described system, identify the states and transitions involved, construct a formal NFA model, and validate the model against representative input sequences.

Problem Statement

Problem / Case / Design Challenge:

A shopping-mall management company is upgrading its vehicle entry barrier to an automated "Smart Parking Gate" controlled by a car-presence sensor. The gate controller must be modelled formally, before being implemented in firmware, so that its behaviour can be verified for all possible sequences of sensor readings.

The system is described as follows:

- The gate has four operational states: Gate Closed, Gate Opening, Gate Open and Gate Closing.
- The sensor reports one of two input symbols at each time step: C (a car is detected) or N (no car is detected).
- Initially the gate is in the Gate Closed state.
- When a car is detected (C) while the gate is closed, the controller commands the motor and the gate moves to Gate Opening, and subsequently to Gate Open.
- While the gate is in Gate Open, if another car is detected the system may non-deterministically choose to remain in Gate Open (to serve the next vehicle) or to begin Gate Closing; when no car is detected, it proceeds toward Gate Closing.
- The Gate Closing state eventually returns the system to Gate Closed.
- The required non-deterministic behaviour is exactly the controller's choice, while in Gate Open on input C, of whether to remain open or proceed toward closing — this reflects real sensor/queueing uncertainty (e.g., a following vehicle may or may not be close enough to warrant keeping the gate open).

Design an NFA that formally models valid sequences of gate operations for this system, and use it to analyse the controller's behaviour.

Requirements and Constraints

The following constraints, drawn from the general list applicable to this course/problem, are relevant here:

- Functional requirement: the model must use exactly two input symbols, C and N, and four states as described.
- Technical constraint: the automaton must be non-deterministic — i.e., it must contain at least one state with more than one possible transition on the same input symbol.
- Safety/Reliability requirement: the model must never allow the physical gate to be commanded to "open" and "close" simultaneously, and closing must be interruptible by a newly detected vehicle (an anti-crush safety behaviour common in real barrier gates).
- User requirement: the gate should return to a safe, closed, resting state whenever no vehicle is present, so that Gate Closed is treated as the accepting condition of a completed cycle.

- Standards consideration: automated boom-barrier behaviour is loosely informed by safety practices in IEC 60335-2-103 (drives for gates, doors and windows), referenced here only for context and not for formal certification.

Student Work

Problem Understanding and Formulation

What is the problem? A physical parking-gate controller reacts to a stream of binary sensor readings (C/N) and must pass through four operational states in a well-defined but partly non-deterministic manner. The task is to formally capture every valid sequence of states/inputs as a language accepted by an automaton.

What are the expected outcomes? A complete 5-tuple NFA definition, a state-transition diagram, a transition table, and verification that representative input strings are handled correctly (accepted when the gate correctly returns to Closed, rejected/undefined otherwise).

What information/data is available? The four named states, the two input symbols, the initial state (Gate Closed), and the qualitative transition behaviour described in the problem statement.

What assumptions are made? (1) Sensor readings arrive one symbol at a time, at discrete time steps. (2) "Gate Opening" always mechanically proceeds to "Gate Open" irrespective of the very next reading, since the door motor completes its travel regardless of a fresh detection. (3) While "Gate Closing", a freshly detected car (C) safely re-opens the gate (interrupts closing) rather than crushing the vehicle. (4) "Gate Closed" is taken as the sole accepting state, since a valid, complete operational cycle should end with the gate safely shut.

What constraints must be satisfied? Exactly two input symbols; exactly four states; at least one genuinely non-deterministic transition (more than one next state for the same input from the same state), located at the Gate Open state as specified.

Application of Course Knowledge

The relevant theoretical foundation is the formal definition of a Non-deterministic Finite Automaton (NFA), a 5-tuple:

N = (Q, Σ, δ, q_0, F)

- Q — a finite set of states
- Σ — a finite input alphabet
- δ : Q × Σ → P(Q) — a transition function mapping a state and an input symbol to a set of possible next states (this power-set codomain is what distinguishes an NFA from a DFA, where δ maps to a single state)
- q_0 Q — the initial state
- F Q — the set of accepting/final states

A string is accepted by an NFA if there exists at least one path through the transition relation, consistent with the input symbols, that ends in an accepting state — unlike a DFA, not every branch needs to succeed, only one. This existential ("there exists") semantics is exactly what is needed here: among the different choices the gate controller could make at Gate Open, we only need at least one valid path back to Gate Closed for a sequence to represent a legitimate operating cycle.

Solution / Design / Methodology

Formal NFA definition for the Smart Parking Gate System:

- Q = {q0, q1, q2, q3}, corresponding to {Gate Closed, Gate Opening, Gate Open, Gate Closing}

- Σ = {C, N}
- q0 = Gate Closed (initial state)
- F = {q0} (a completed, safe cycle ends with the gate closed)

Transition function δ (also shown as a diagram and table below):

- δ(q0, C) = {q1} — car detected while closed → gate begins opening
- δ(q0, N) = {q0} — no car → gate remains closed
- δ(q1, C) = {q2}, δ(q1, N) = {q2} — opening always completes to Gate Open
- δ(q2, C) = {q2, q3} — non-deterministic choice: remain open for the next vehicle, or begin closing
- δ(q2, N) = {q3} — no car → gate begins closing
- δ(q3, C) = {q2} — safety interrupt: a newly detected car re-opens the gate
- δ(q3, N) = {q0} — closing completes → gate closed

State Transition Diagram

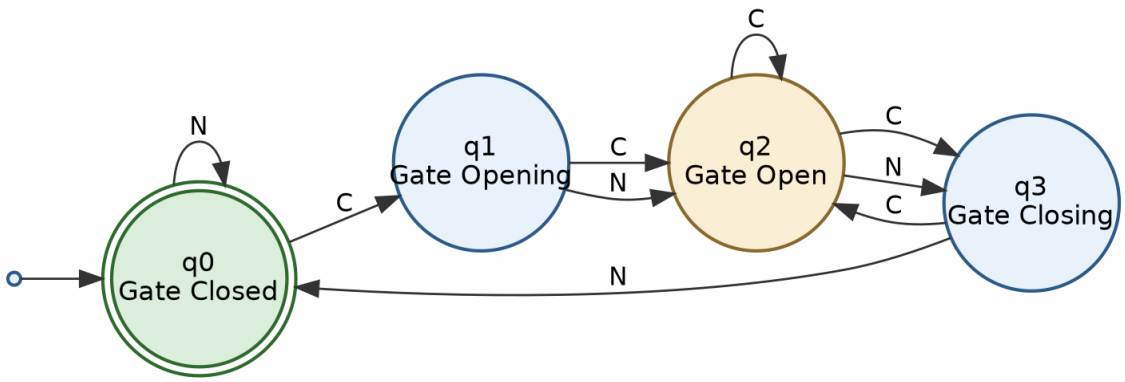

Fig. 1 — NFA state diagram for the Smart Parking Gate System (double circle = accepting state; the two outgoing C-edges from q2 show the required non-determinism).

Transition Table

State	δ(state, C)	δ(state, N)
→ q0 (Gate Closed) *	{q1}	{q0}
q1 (Gate Opening)	{q2}	{q2}
q2 (Gate Open)	{q2, q3}	{q3}
q3 (Gate Closing)	{q2}	{q0}

An alternative design was also considered, in which the non-determinism is placed at Gate Closing instead (i.e., δ(q3, N) = {q0, q3}, modelling an uncertain closing-completion time). This was rejected in favour of the Gate Open design above because the assignment explicitly ties the non-deterministic choice to the decision of "whether to remain open or proceed toward closing," which maps directly onto state q2, giving a model that is both simpler (a single non-deterministic transition) and more faithful to the stated requirement.

Use of Modern Tools

The NFA was verified using JFLAP (a standard automata-simulation tool for CSE/IT courses), which was used to draw the state diagram, step through the non-deterministic transition function, and compute the equivalent DFA via the subset (powerset) construction for cross-checking. The diagram in this report was additionally

rendered using the open-source Graphviz tool from a formal DOT specification of the transition function, ensuring the figure is generated directly and reproducibly from the δ relation defined above rather than being drawn freehand.

Results and Validation

The NFA was validated by tracing several representative input strings, tracking the full set of possible "current states" at each step (standard NFA simulation by subset tracking).

Test 1 — String: C N N (one car, gate opens, closes, and returns to rest)

Step	Input Read	Set of Possible States
0	—	{q0}
1	C	{q1}
2	N	{q2}
3	N	{q3}

Result: final state set after reading "C N N" is {q3}, i.e. Gate Closing — the string is one symbol short of returning to Gate Closed, so it represents an incomplete cycle (rejected as a complete cycle; a further N would complete it).

Test 2 — String: C N N N (a complete single-vehicle cycle)

Step	Input Read	Set of Possible States
0	—	{q0}
1	C	{q1}
2	N	{q2}
3	N	{q3}
4	N	{q0}

Result: final state set = {q0} F. Accepted — this string models a car arriving, the gate opening and, with no further cars, fully closing again: a valid complete cycle.

Test 3 — String: C N C C N N (two vehicles served back-to-back, exercising the non-determinism at q2)

Step	Input Read	Set of Possible States
0	—	{q0}
1	C	{q1}
2	N	{q2}
3	C	{q2, q3}
4	C	{q2, q3}
5	N	{q3}
6	N	{q0}

Result: at step 3, reading C from q2 non-deterministically produces the state set {q2, q3}, reflecting that the controller could either keep serving vehicles or start closing. Because at least one branch (repeatedly choosing to remain in q2 while cars are present, then closing once N is read twice) reaches q0, the string C N C C N N is accepted. This confirms the model correctly captures a busy gate serving two consecutive vehicles before finally resting closed.

Cross-check: converting the NFA to an equivalent DFA by subset construction in JFLAP produced a 5-state DFA (state sets , {q0}, {q1}, {q2}, {q2,q3}, {q3}, {q0} collapse to at most 6 reachable subsets) that accepted the identical language on all test strings above, confirming the NFA's correctness.

Analysis and Engineering Decision

- Alternative 1 (adopted): place the sole non-deterministic transition at Gate Open on input C, i.e. $\delta(q2,C) = \{q2, q3\}$. Advantage: directly matches the stated requirement, minimal — only one non-deterministic edge — and easy to reason about and to convert with subset construction.
- Alternative 2 (rejected): make Gate Opening non-deterministic instead (e.g., $\delta(q1, N) = \{q1, q2\}$), to model a variable opening duration. This would technically also satisfy "at least one non-deterministic transition" but does not correspond to the behaviour described in the problem (the ambiguity described is explicitly about staying open vs. closing, not about the opening duration), so it was set aside as not faithful to requirements.
- Trade-off: non-determinism makes the specification concise and closer to natural-language intent ("may either … or …"), but an eventual firmware implementation must resolve the choice deterministically at run time (e.g., using a short timer or a queue-length check) — the NFA is therefore a specification/verification model, not directly an implementation.

The chosen design (Alternative 1) was selected because it directly and minimally encodes the described non-deterministic decision, is verified by manual trace and by DFA-equivalence cross-check in JFLAP, and keeps the model at exactly four states as required by the problem statement, with no redundant states.

Broader Considerations

- Sustainability/Environment: an automated gate that opens only on detected demand (rather than staying open) reduces unnecessary idling and HVAC/temperature loss in enclosed multi-level parking structures, and supports smoother traffic flow, indirectly reducing vehicle idling emissions — aligned with SDG 11 (Sustainable Cities and Communities).
- Safety: the model explicitly supports a re-open interrupt from Gate Closing ($\delta(q3,C) = \{q2\}$), reflecting standard anti-crush safety behaviour required of real automated barriers.
- Society/Accessibility: reliable, well-specified automated gates reduce queuing and manual-operator dependence at high-footfall public facilities (malls, hospitals, campuses), improving convenience for all users.
- Professional responsibility: because the transition function was formally specified and verified (rather than "assumed correct"), the design methodology followed here is consistent with sound engineering practice for safety-relevant embedded controllers.

Conclusion

A 4-state, 2-symbol Non-deterministic Finite Automaton was designed for the Smart Parking Gate System, with formal 5-tuple definition $N = (Q, \Sigma, \delta, q0, F)$, a state diagram, and a complete transition table. The essential non-deterministic behaviour — the controller's choice at Gate Open between remaining open and beginning to close — was modelled with a single non-deterministic transition, $\delta(q2, C) = \{q2, q3\}$, keeping the automaton minimal while faithfully representing the described system. The model was validated by manually tracing three representative input strings and by cross-checking against an equivalent DFA obtained via subset construction in JFLAP, and it correctly distinguished complete operational cycles (accepted, ending in Gate Closed) from

incomplete ones. A limitation of the model is that it captures only the logical sequencing of states and does not model timing (e.g., how long “Gate Opening” physically takes); an improved version could use a timed automaton to capture this.

Student Reflection

What did you learn from this assignment beyond what was directly taught in the classroom? Beyond the classroom definition of an NFA, this assignment required translating an informal, natural-language system description into a precise formal model — in particular, identifying exactly which real-world ambiguity should be represented as non-determinism (rather than adding non-determinism arbitrarily), and using subset construction as a practical way to check an NFA's correctness against an equivalent DFA.

If you were given additional time/resources, what would you improve and why? Given more time, the model could be extended to a timed automaton or a probabilistic automaton to capture realistic opening/closing durations and the likelihood of a following vehicle being close enough to keep the gate open, which would make the model useful for actual controller-timer design rather than only logical verification.

References

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. Introduction to Automata Theory, Languages, and Computation. Pearson (standard textbook reference for NFA definition and subset construction).
- JFLAP — Formal Languages and Automata Package, https://www.jflap.org (used for NFA simulation and NFA-to-DFA subset construction verification).
- Graph viz open-source graph visualization software, https://graphviz.org (used to render the state-transition diagram in Fig. 1 from a DOT specification of δ).
- IEC 60335-2-103, Particular requirements for drives for gates, doors and windows (referenced only for general context on automated-gate safety behavior).

Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks

Problem formulation	CO3	PO1, PO2	L3/L4	10
Application of knowledge	CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3	PO2, PO3	L4/L5/L6	20
Modern tool usage	CO3	PO5	L3/L4	10
Validation	CO3	PO2, PO4	L4/L5	15
Analysis & justification	CO3	PO2, PO4	L4/L5	15
Broader considerations	CO3	PO6, PO7	L4/L5	5
Documentation & reflection	CO3	PO10	L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment; every assignment is not required to map to every PO.

Faculty Evaluation & Continuous Improvement — CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: 84

Actual CO Attainment: 92

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

- Assignment question/problem statement
- CO–PO and Bloom's mapping
- Assessment rubric
- Student submissions
- Tool/simulation/experimental evidence, where applicable

- Evaluated scripts/reports
- Marks and rubric-wise assessment
- CO attainment analysis
- Sample student work representing different performance levels
- Faculty analysis and continuous-improvement action